TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI (DNTU)
KHOA CÔNG NGHỆ THÔNG TIN – HỌC PHẦN WEB API (TH71108)

VISIONARY

HỆ THỐNG QUẢN LÝ HOẠT ĐỘNG HỘI NHẬP & KẾT NỐI SINH VIÊN

Mô hình Một Web API (ASP.NET Core 8) – Ba Ứng dụng Khách (Web Next.js, Desktop WinForms, Mobile Android)

1. Quách Khoa Đức
MSSV: 1923040025
Lead, Kiến trúc hệ thống & Web Client (Next.js)
🎯 **Phần 1: Dẫn nhất, mở đầu & Web Client Demo**

2. Đào Trí Khang
MSSV: 1923040156
Mobile Software Engineer (Android Native Java)
🎯 **Phần 2: Ứng dụng Android, Threading & Demo App**

3. Hồ Đoàn Nguyên Khang
MSSV: 1923040112
API Engineer & Database (ASP.NET Core & Swagger)
🎯 **Phần 3: Swagger, 6 chặng, JWT 401/403 & DB DTO**

4. Võ Ngọc Minh Đạt
MSSV: 1923040088
Desktop Software Engineer (WinForms C#) & BCN CLB
🎯 **Phần 4: WinForms, Sự cố CI, So sánh & KẾT THÚC**

🌐 **THÔNG TIN TRUY CẬP VÀ XÁC THỰC HỆ THỐNG:**

- Swagger UI (Kiểm thử API trực tiếp): https://visionary-api-gb51.onrender.com/swagger
- Website Production (Vercel): https://visionary-platform.vercel.app | Mã nguồn: https://github.com/lovemvaf/visionary-api
- Khai báo sử dụng AI: Google Antigravity hỗ trợ kiểm chứng mã nguồn & kiểm thử hồi quy; toàn bộ logic nghiệp vụ do nhóm trực tiếp làm chủ.

BÀI TOÁN THỰC TẾ & MỤC TIÊU CỦA ĐỀ TÀI VISIONARY

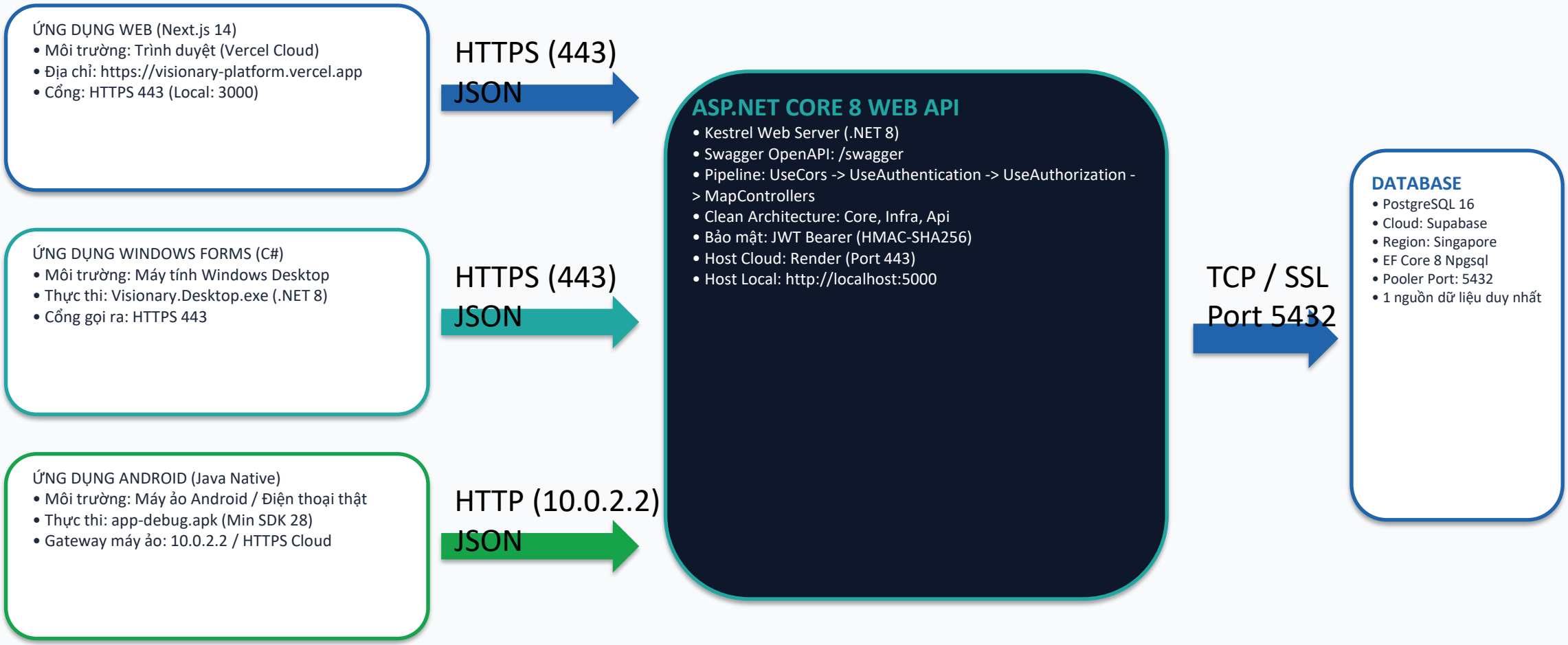
VẤN ĐỀ THỰC TẾ CỦA CLB & SINH VIÊN

1. Quản lý thủ công, rời rạc:
CLB Tiếng Anh DNTU trước đây quản lý qua Google Form và file tính, dễ nhầm lẫn dữ liệu và thất lạc thông tin người tham gia.
2. Thiếu cổng thông tin hội nhập & cơ hội:
Sinh viên CNTT thiếu kênh tiếp cận học thuật English for IT, Radar công nghệ mở (GitHub/Hugging Face) và các cuộc thi Hackathon.
3. Trải nghiệm không đồng bộ trên các thiết bị:
Sinh viên dùng điện thoại di động, Ban Chủ Nhiệm quản trị trên máy tính Windows, độc giả truy cập trên trình duyệt Web.

MỤC TIÊU GIẢI PHÁP VISIONARY

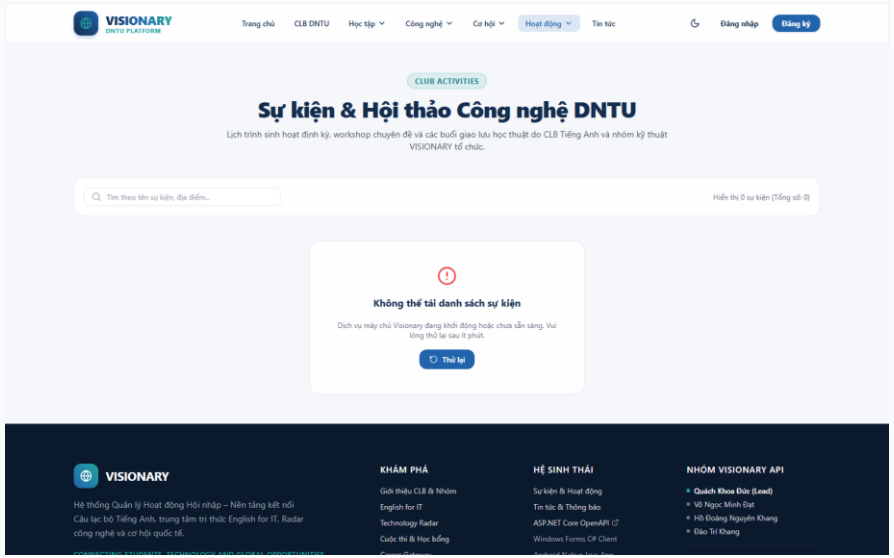
1. Xây dựng một Web API trung tâm duy nhất:
ASP.NET Core 8 Web API làm hạt nhân xử lý toàn bộ logic nghiệp vụ, xác thực JWT và kết nối cơ sở dữ liệu PostgreSQL.
2. Phục vụ ba ứng dụng khách chuyên biệt:
- Web Next.js: Cổng thông tin công khai, tin tức hội nhập, tra cứu sự kiện, English for IT.
- Windows Forms C#: Phần mềm quản trị cho BCN CLB: quản lý thành viên, duyệt đơn, điểm danh sự kiện.
- Android Native Java: Ứng dụng di động cho sinh viên: xem sự kiện, đăng ký tham gia, nhận mã QR check-in.
3. Đảm bảo toàn vẹn dữ liệu (Data Integrity):
Không dùng dữ liệu mẫu (mock/fallback), bảo vệ bài nháp, phân quyền chặt chẽ theo vai trò (RBAC).

SƠ ĐỒ KIẾN TRÚC HỆ THỐNG: MỘT WEB API, BA NỀN TẢNG



💡 **TẠI SAO DÙNG CHUNG MỘT API? Đảm bảo tính nhất quán dữ liệu 100%, tập trung logic kiểm tra quyền trên server, không phân mảnh dữ liệu giữa các client.**

ỨNG DỤNG WEB CLIENT: GIAO DIỆN & KẾT NỐI API



Tên điều khiển	Loại điều khiển	Vai trò nghiệp vụ
txtSearchEvents	HTML <input type='text'>	Nhập từ khóa tìm kiếm sự kiện
cboEventCategory	HTML <select>	Lọc sự kiện theo danh mục học thuật
btnSearch	HTML <button>	Kích hoạt gửi request GET /api/events
grdEventCards	HTML (Grid layout)	Hiển thị danh sách thẻ sự kiện do API trả về

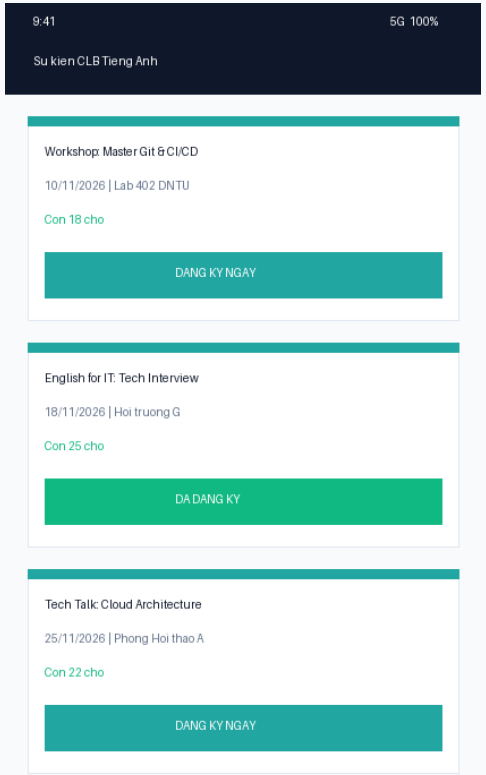
// frontend/src/services/eventService.ts – Gọi GET /api/events (Mã nguồn thật):

```
export const getEvents = async (page = 1, pageSize = 10, category?: string) => {
  const params = new URLSearchParams({ page: page.toString(), pageSize: pageSize.toString()
});
  if (category) params.append('category', category);

  // Goi API qua ham fetch tap trung, tu dong truyen Bearer token neu co
  const response = await fetchWithAuth(`/api/events?${params.toString()}`, {
    method: 'GET'
  });
  return response; // Tra ve ApiResponse<PagedResponse<EventDto>>
};
```

Khó khăn đặc thù Web & Giải pháp: Trình duyệt ép kiểm tra CORS. Giải pháp: Cấu hình app.UseCors() trên ASP.NET Core và triển khai BFF Proxy route tại /api/[...path].

ỨNG DỤNG DI ĐỘNG: ANDROID NATIVE JAVA



Tên điều khiển	Loại điều khiển	Vai trò nghiệp vụ
lstEvents	RecyclerView	Danh sách cuộn các sự kiện CLB
btnRegister	Button	Đăng ký tham gia sự kiện trực tiếp
swpRefresh	SwipeRefreshLayout	Vuốt xuống để tải lại dữ liệu mới nhất
txtSearch	EditText	Ô tìm kiếm sự kiện theo từ khóa

```
// EventsFragment.java – Gọi API bất đồng bộ qua Retrofit (Mã nguồn thật):
ApiClient.getService(getContext()).getEvents(1, 20)
    .enqueue(new Callback<ApiResponse<PagedResponse<EventItem>>>() {
        @Override
        public void onResponse(@NonNull Call<ApiResponse<PagedResponse<EventItem>>> call,
                               @NonNull Response<ApiResponse<PagedResponse<EventItem>>> res) {
            swipeRefresh.setRefreshing(false);
            if (res.isSuccessful() && res.body() != null && res.body().isSuccess()) {
                adapter.setEvents(res.body().getData().getItems());
            }
        }
    });
```

Khó khăn đặc thù Android & Giải pháp: Android 9+ chặn kết nối HTTP không mã hóa và máy ảo không dùng localhost. Nhóm cấu hình network_security_config.xml cho phép cleartext traffic trong dev và dùng alias 10.0.2.2.

SƠ ĐỒ CƠ CHẾ MẠNG & ĐIỀU PHỐI LUỒNG TRÊN ANDROID

1. Tương tác UI
[Main UI Thread]
Người dùng nhấn btnRegister
hoặc vuốt làm mới swpRefresh

→

2. Retrofit Enqueue
[OkHttp Dispatcher]
apiService.getEvents().enqueue()
Đẩy việc vào luồng nền Worker

→

3. Gói tin Mạng
[I/O Network TCP]
Gửi request GET /api/events
tới Kestrel Server cổng 443

→

4. Giải mã JSON
[Gson Deserializer]
Tự động ánh xạ JSON string
sang ApiResponse<PagedResponse>

→

5. Cập nhật UI
[Main UI Thread]
Handler.post / onResponse
Cập nhật Adapter cho View

// EventsFragment.java – Hai dòng mã cốt lõi: Tạo luồng nền & Đẩy kết quả về Luồng Chính (Mã nguồn thật):

```
// 1. TẠO LUỒNG NỀN: .enqueue() đẩy tác vụ mạng sang OkHttp ThreadPool, tránh chặn Main UI Thread
apiService.getEvents(1, 10).enqueue(new Callback<ApiResponse<PagedResponse<EventItem>>>() {
    @Override public void onResponse(Call<...> call, Response<ApiResponse<PagedResponse<EventItem>>> res) {
        if (res.isSuccessful() && res.body() != null) {
            // 2. ĐẨY VỀ LUỒNG CHÍNH: Looper.getMainLooper() đưa dữ liệu về Main UI Thread để cập nhật Adapter an toàn
            new Handler(Looper.getMainLooper()).post(() -> eventAdapter.setEvents(res.body().getData().getItems()));
        }
    }
});
```

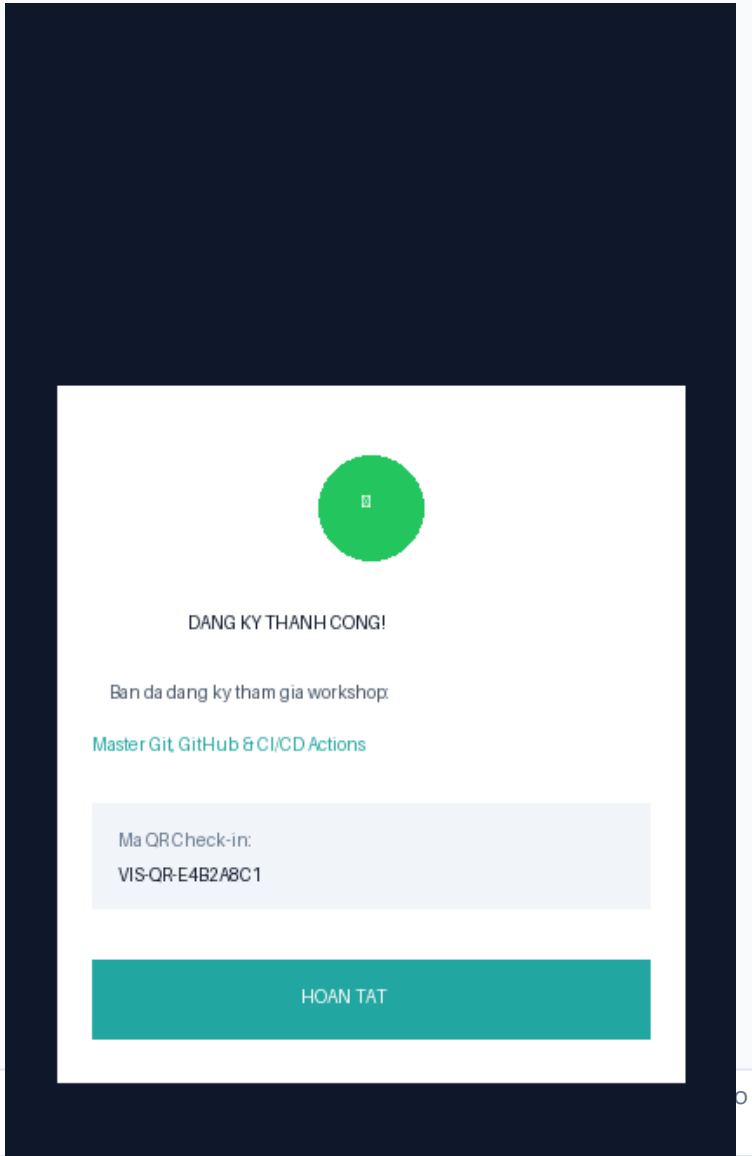
⚠️ HAI NGOẠI LỆ NGUY HIỂM BUỘC PHẢI TRÁNH:

1. NetworkOnMainThreadException: Xảy ra khi gọi mạng trực tiếp (.execute()) trên Luồng Chính. Hệ thống Android tự động chặn và ném lỗi crash ngay.
2. CalledFromWrongThreadException: Xảy ra khi cố tình sửa đổi View từ luồng nền. Bắt buộc phải đưa về Main Thread qua Handler(Looper.getMainLooper()).

🌐 CẤU HÌNH KẾT NỐI MẠNG ĐẶC THÙ TRÊN ANDROID:

- Địa chỉ 10.0.2.2: Máy ảo Android không dùng 'localhost' vì localhost trỏ về chính nó. 10.0.2.2 là địa chỉ bí danh trỏ về máy tính chủ chạy Web API.
- Khai báo usesCleartextTraffic: Từ Android 9, kết nối HTTP không mã hóa bị chặn. Nhóm cấu hình network_security_config.xml cho môi trường test.
- Tự động gắn JWT qua AuthInterceptor: Token lưu trong SharedPreferences và được tự động chèn vào header Authorization.

DEMO TRỰC TIẾP: ĐĂNG KÝ SỰ KIỆN TRÊN ANDROID APP



KỊCH BẢN THAO TÁC DEMO TRỰC TIẾP TRÊN MÁY ẢO:

👉 **Bước 1: Mở ứng dụng Android (app-debug.apk đã nạp sẵn)**
Ứng dụng tự động gửi request GET /api/events nạp danh sách sự kiện lên RecyclerView.

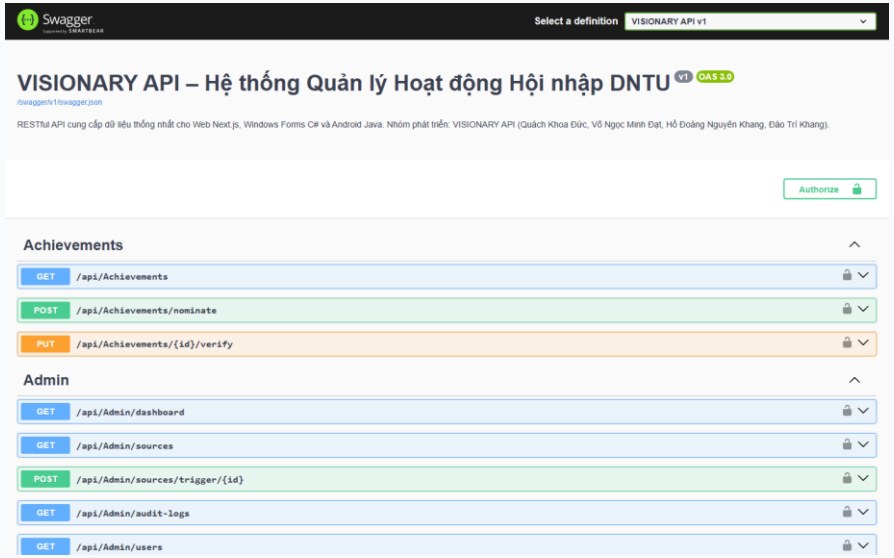
👉 **Bước 2: Chọn sự kiện 'Hội thảo Kỹ năng Hội nhập Quốc tế'**
Màn hình chuyển sang EventDetailActivity hiển thị chi tiết thời gian, địa điểm, diễn giả.

👉 **Bước 3: Nhấn nút 'Đăng ký tham gia' (btnRegister)**
Ứng dụng gửi yêu cầu POST /api/events/{id}/register kèm Bearer Token của tài khoản sinh viên.

👉 **Bước 4: Quan sát phản hồi HTTP 200 từ Web API**
Server tiếp nhận, ghi bản ghi vào PostgreSQL và trả về JSON thành công kèm mã check-in.

👉 **Bước 5: Bàn giao dữ liệu cho WinForms**
Bản ghi vừa đăng ký sẽ được bạn Võ Ngọc Minh Đạt tải và điểm danh ngay trên ứng dụng Windows Forms.

SWAGGER UI & BẢNG HỢP ĐỒNG API THỰC TẾ TIÊU THỤ



Method	Đường dẫn (Endpoint)	Quyền	Thân yêu cầu	Mã trả về
POST	/api/auth/login	Mở	username, password	200 + Token
GET	/api/events	Mở	(query phân trang)	200 + Items
GET	/api/events/{id}	Mở	id trên URL	200 / 404
POST	/api/events	Admin/BCN	title, time, location	201 Created
POST	/api/events/{id}/register	Student	Header Bearer Token	200 OK / 400

📌 **Tên trường JSON tuân thủ tuyệt đối chuẩn camelCase (id, title, location, totalCount, items). Khớp 100% giữa ASP.NET Core và cả 3 Client.**

SƠ ĐỒ CHU TRÌNH 6 CHẶNG CỦA MỘT YÊU CẦU: GET /api/events

CHẶNG 1: THAO TÁC NGƯỜI DÙNG
[Luồng Giao diện [UI Thread]]
- Người dùng kích hoạt hành động trên màn hình
- Web Next.js: Bấm tab 'Sự kiện' / onClick
- WinForms C#: Sự kiện btnTaiDanhSach_Click
- Android Java: onResume() hoặc vuốt swpRefresh

CHẶNG 2: CLIENT XỬ LÝ & GỌI MẠNG
[Bất đồng bộ [Async Worker]]
- Client chuẩn bị tham số & tạo HTTP request
- Web: useEffect() -> api.getEvents(1, 10)
- WinForms: TaiDanhSachAsync() -> _http.GetAsync()
- Android: loadEvents() -> api.getEvents().enqueue()

CHẶNG 3: TRUYỀN GÓI TIN QUA MẠNG
[Đường truyền Mạng [TCP/SSL]]
- Gói tin HTTP chuẩn gửi qua mạng tới cổng 443:
  GET /api/events?pageNumber=1&pageSize=10
- Host: visionary-api-gb51.onrender.com:443
- Headers: Accept: application/json; charset=utf-8

CHẶNG 4: KESTREL SERVER & DATABASE
[ASP.NET Core 8 & PostgreSQL]
- Middleware: UseCors -> UseAuth -> MapControllers
- EventsController.GetEvents() nhận request
- EventService truy vấn PostgreSQL qua EF Core:
  _context.Events.AsNoTracking().ToPagedListAsync()

CHẶNG 5: ĐÓNG GÓI PHẢN HỒI JSON
[Tuần tự hóa [System.Text.Json]]
- Chuyển Entity -> EventDto (ẩn trường nhạy cảm)
- Đóng gói ApiResponse<PagedResponse<EventDto>>
- Trả về HTTP 200 OK với body JSON chuẩn camelCase:
  {"success": true, "data": {"items": [...], ...}}

CHẶNG 6: NHẬN JSON & CẬP NHẬT UI
[Luồng Giao diện [UI Thread]]
- Client giải mã JSON và đưa về Main UI Thread:
- Web: setEvents(data.items) -> Virtual DOM render
- WinForms: grdEvents.DataSource = items (UI)
- Android: runOnUiThread -> adapter.notifyDataSetChanged()

🎯 **ĐIỂM CỐT LÕI: Cả 3 nền tảng đều đi qua đúng 6 chặng này, sử dụng chung hợp đồng API, chung DTO và chung nguồn cơ sở dữ liệu.**

SƠ ĐỒ VÒNG ĐỜI JWT, XÁC THỰC & PHÂN BIỆT 401 VS 403

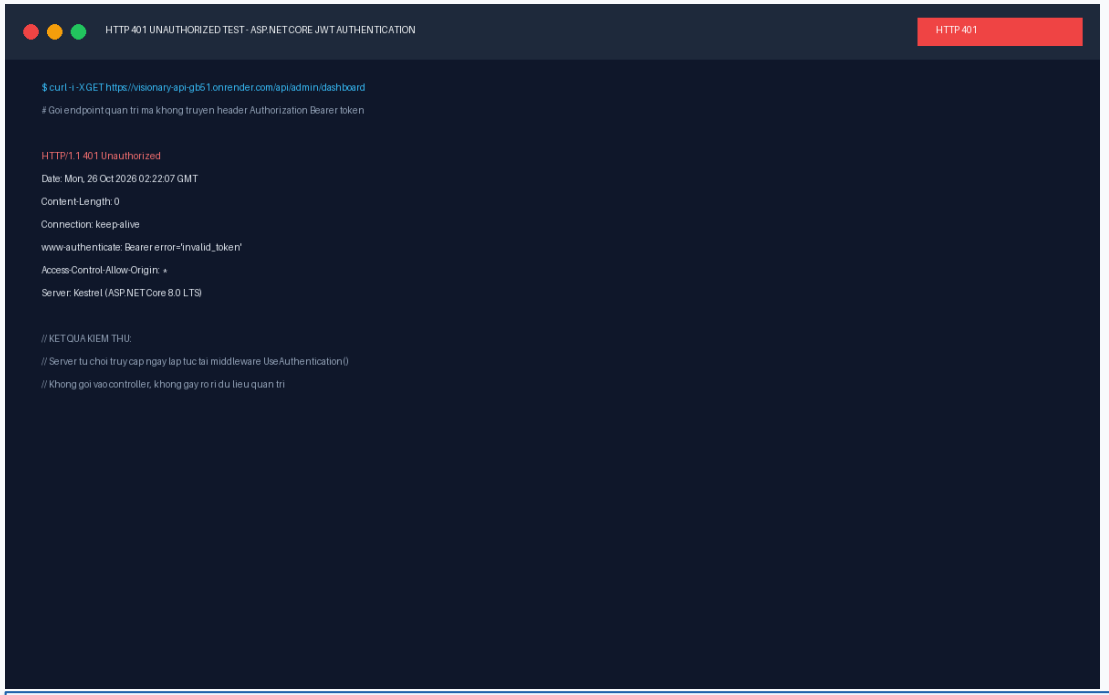
1. Đăng nhập
POST /api/auth/login
Gửi username & password

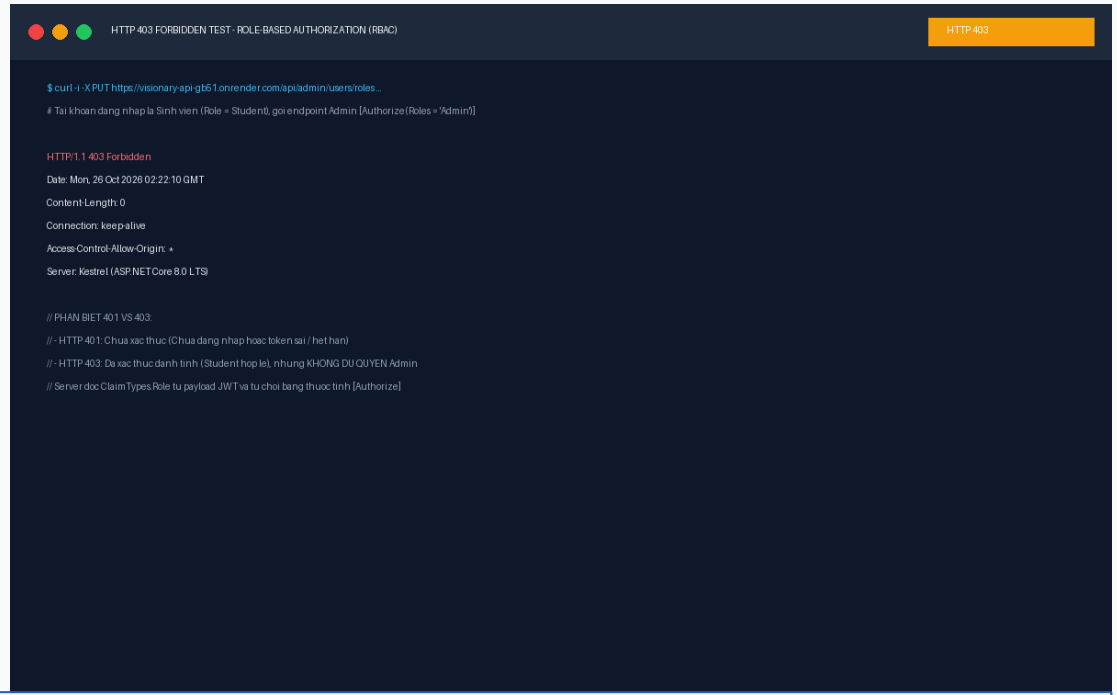
2. Phát JWT Token
Ký HMAC-SHA256
Chứa claims UserId & Role

3. Lưu trữ Client
Client Storage
Web: LocalStorage
Android: SharedPreferences
WinForms: AppState

4. Gọi API cần Auth
Bearer Header
Gửi header:
Authorization: Bearer <token>

5. Server Kiểm tra
Middleware RBAC
Giải mã token, kiểm tra role:
• 401: Thiếu / Hết hạn
• 403: Sai vai trò





🔒 **NGUYÊN TẮC: 401 Unauthorized (Chưa đăng nhập / Token hết hạn) khác hẳn 403 Forbidden (Đã đăng nhập nhưng không đủ quyền). Server tự đọc UserId từ Claims, tuyệt đối không tin tham số client gửi lên.**

JSON, DTO, PHÂN TRANG & THIẾT KẾ CƠ SỞ DỮ LIỆU

```
// Phản hồi JSON thực tế (camelCase) từ GET /api/events:
{
  "success": true,
  "data": {
    "items": [{ "id": 1, "title": "Workshop: Git & CI/CD", "category": "Tech" }],
    "pageNumber": 1, "pageSize": 10, "totalCount": 24, "totalPages": 3
  }
}
```

```
// PagedResponse.cs – Vỏ bọc phân trang chuẩn C# (Mã nguồn thật):
public class PagedResponse<T> {
    public List<T> Items { get; set; } = new();
    public int TotalCount { get; set; }
    public int PageNumber { get; set; }
    public int PageSize { get; set; }
    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);
    public bool HasNextPage => PageNumber < TotalPages;
}
```

🛡️ VAI TRÒ CỐT LÕI CỦA DTO TRONG HỆ THỐNG:

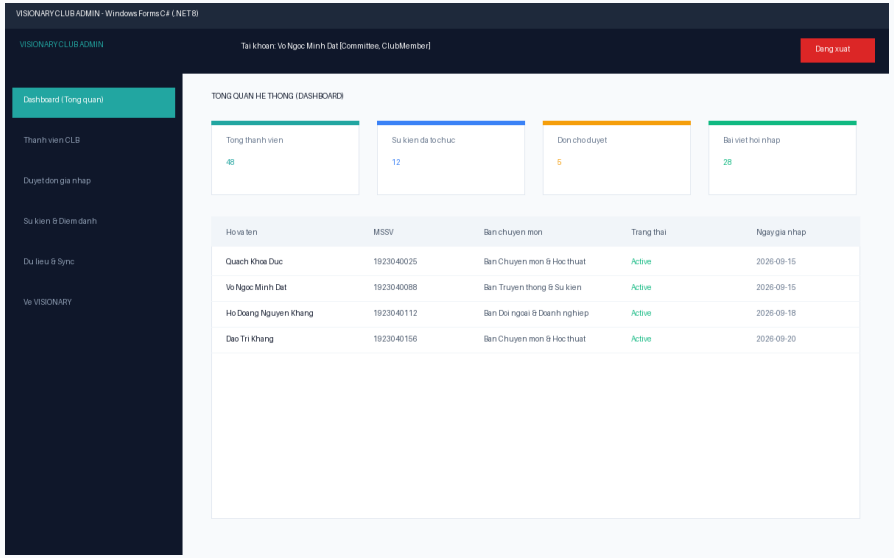
- Không bao giờ trả Entity trực tiếp: Che giấu thông tin nhạy cảm như PasswordHash và trường kiểm toán nội bộ.
- Chống đệ quy vòng lặp JSON (Cyclic Reference) giữa các bảng có quan hệ 1-N trong Entity Framework Core.
- Tối ưu kích thước gói tin mạng (Payload Size), chỉ gửi những trường giao diện thực sự cần.

🗄️ THIẾT KẾ DATABASE & TỐI ƯU EF CORE 8:

- PostgreSQL 16 trên Supabase: Chuẩn ACID, hỗ trợ JSONB, Pooler cổng 5432 giúp phục vụ nhiều kết nối đồng thời.
- Tối ưu AsNoTracking(): Sử dụng cho toàn bộ các API truy vấn đọc dữ liệu, tăng tốc độ phản hồi 30-40%.
- Khóa ngoại (Foreign Keys): Ràng buộc toàn vẹn giữa Users, Roles, Events và Registrations, chống dữ liệu mồ côi.

🎯 **NGUYÊN TẮC PHÂN TRANG: Luôn phân trang ở tầng Database qua LINQ .Skip().Take(). Giới hạn trần Math.Min(pageSize, 50) ngăn chặn client ép server quá tải.**

ỨNG DỤNG QUẢN TRỊ DESKTOP: WINDOWS FORMS C#



Tên điều khiển	Loại điều khiển	Vai trò nghiệp vụ
grdDangKy	DataGridView	Hiển thị bảng danh sách sinh viên đăng ký sự kiện
btnDiemDanh	Button	Xác nhận điểm danh sinh viên tham gia sự kiện
cboSuKien	ComboBox	Chọn sự kiện cần xem danh sách đăng ký
lblTongSo	Label	Hiển thị tổng số lượt đăng ký từ server trả về

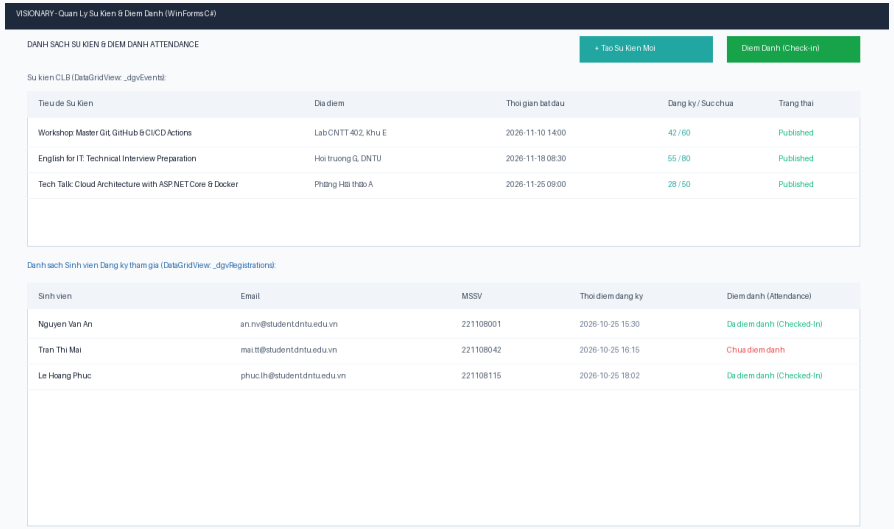
// ApiService.cs – Tái sử dụng HttpClient & async/await (Mã nguồn thật):

```
public class ApiService {
    // Dung chung HttpClient duy nhat (Singleton) de tranh can kiet socket
    private static readonly HttpClient _http = new HttpClient();

    public async Task<ApiResponse<T>> GetAsync<T>(string endpoint) {
        using var req = new HttpRequestMessage(HttpMethod.Get, _baseUrl + endpoint);
        if (!string.IsNullOrEmpty(_token)) req.Headers.Authorization = new("Bearer", _token);
        var res = await _http.SendAsync(req); // await tra quyen dieu khien cho UI
        return await res.Content.ReadFromJsonAsync<ApiResponse<T>>();
    }
}
```

Quy tắc sống còn trên WinForms: Tuyệt đối không gọi .Result hoặc .Wait() trên Task vì sẽ gây DEADLOCK khóa cứng giao diện. Luôn dùng async và await.

DEMO TRỰC TIẾP: QUẢN TRỊ VÀ ĐIỂM DANH TRÊN WINFORMS



KỊCH BẢN THAO TÁC DEMO ĐIỂM DANH TRỰC TIẾP:

👉 **Bước 1: Mở ứng dụng Visionary.Desktop.exe**
Phần mềm đã được biên dịch sẵn sang .NET 8, mở ra là chạy ngay.

👉 **Bước 2: Đăng nhập tài khoản Ban Chủ Nhiệm (Committee)**
Gọi POST /api/auth/login, nhận JWT token và lưu vào biến tĩnh AppState.Token.

👉 **Bước 3: Chọn sự kiện 'Hội thảo Kỹ năng Hội nhập Quốc tế'**
Gọi GET /api/events/{id}/registrations để nạp danh sách đăng ký lên DataGridView.

👉 **Bước 4: Kiểm chứng sinh viên vừa đăng ký trên Android**
Thấy ngay bản ghi của bạn Trí Khang vừa bấm đăng ký ở phần demo Android trước.

👉 **Bước 5: Nhấn nút 'Xác nhận Điểm danh' (btnDiemDanh)**
Gọi POST /api/events/{id}/attendance, server cập nhật trạng thái 'Đã tham gia' tức thì.

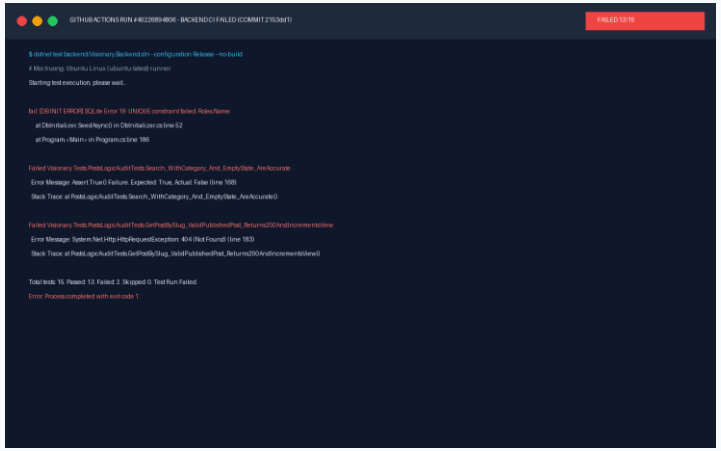
BA ĐIỂM KHÁC BIỆT KỸ THUẬT & ĐIỀU BẮT BUỘC GIỐNG NHAU

Khác biệt Kỹ thuật	Web Client (HTML/TS)	Windows Forms (C#)	Android Native (Java)
1. Thư viện mạng &	fetch() API	HttpClient	Retrofit 2 + OkHttp
Bất đồng bộ	Promise & async/await Event Loop trình duyệt	Task & async/await SynchronizationContext	Call.enqueue() Callback Worker Thread chạy ngầm
2. Điều phối Luồng	Tự động hoàn toàn	Tự động quay về UI Thread	Bắt buộc chuyển thủ công:
giao diện (UI Thread)	(Single-threaded JS Engine không lo xung đột luồng)	sau lệnh await (Tuyệt đối tránh .Result)	Handler(Looper.getMainLooper()) để cập nhật View
3. Môi trường chạy &	Trình duyệt (Browser)	Desktop Windows OS	Thiết bị di động Android
Quản lý JWT Token	Lưu trong localStorage / Cookie	Lưu trong biến static AppState	Lưu SharedPreferences bền vững

⭐ **NHỮNG ĐIỀU BẮT BUỘC PHẢI GIỐNG HỆT NHAU TRÊN CẢ BA NỀN TẢNG:**

1. Hợp đồng API (API Contract): Cùng phương thức HTTP (GET, POST, PUT), cùng đường dẫn (/api/events), cùng tên trường JSON camelCase.
2. Xác thực và Phân quyền: Cùng cấu trúc Header 'Authorization: Bearer <token>', cùng mã lỗi 401 khi hết hạn và 403 khi thiếu quyền.
3. Quy tắc nghiệp vụ (Business Rules): Cùng kiểm tra hạn chót đăng ký trên server, cùng tính số lượng chỗ trống từ cơ sở dữ liệu.

SỰ CỐ THỰC TẾ TRÊN GITHUB ACTIONS CI VÀ QUY TRÌNH SỬA LỖI



SƠ ĐỒ CHẨN ĐOÁN & SỬA LỖI

1. BẮT ĐẦU: CI RUN #12 THẤT BẠI

13 Passed / 2 Failed trên Ubuntu Linux.

- Search test fail dòng 168 (TotalCount = 0)
- GetPostBySlug fail dòng 183 (404 Not Found)

2. NGUYÊN NHÂN GỐC RỄ (ROOT CAUSE)

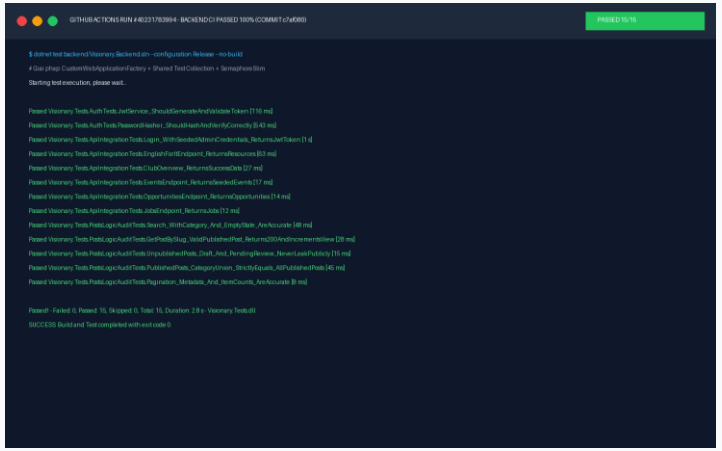
Hai Test Class chạy song song trên xUnit:
Cùng khởi tạo WebApplicationFactory trỏ vào visionary.db -> Đụng độ UNIQUE Roles.Name -> Hủy ngang SeedAsync trước khi nạp Posts!

3. GIẢI PHÁP KỸ THUẬT (REMEDIATION)

- Khóa luồng tĩnh SemaphoreSlim(1,1) trong DbInitializer.SeedAsync().
- Tạo CustomWebApplicationFactory cấp phát DB test cô lập.
- Gộp chung [Collection("IntegrationTests")].

4. KẾT QUẢ KIỂM CHỨNG (VERIFICATION)

Chạy lại kiểm thử hồi quy 3 lần liên tiếp:
15/15 Passed 100% (2.8s) -> Push commit c7af080 lên GitHub -> Run #40231783994 XANH!



✅ **ĐÃ XÁC MINH TRÊN GITHUB ACTIONS: Run #40231783994 (Commit c7af080) hoàn thành xuất sắc – 15/15 Tests Passed 100% trên Linux x64.**

GIỚI HẠN THỰC TẾ, HƯỚNG PHÁT TRIỂN & KẾT LUẬN

BA GIỚI HẠN THỰC TẾ CỦA HỆ THỐNG HIỆN TẠI

1. Chưa có kiểm thử tự động E2E đa nền tảng đồng thời:
   Nhóm mới kiểm thử tự động xUnit Backend và build APK độc lập; chưa có kịch bản E2E tự động xuyên suốt Web -> Mobile -> Desktop.
2. Chưa có thông báo đẩy thời gian thực (Push Notification):
   Khi BCN điểm danh hoặc có sự kiện mới, sinh viên chưa nhận được thông báo qua Firebase Cloud Messaging (FCM) hoặc SignalR.
3. Một chỗ mã nguồn chưa tối ưu:
   Cơ chế Refresh Token tự động khi JWT 4 giờ hết hạn chưa được triển khai hoàn chỉnh, người dùng vẫn phải đăng nhập lại.

KẾ HOẠCH NẾU CÓ THÊM 2 TUẦN NỮA

1. Tích hợp SignalR Hub vào ASP.NET Core:
   Cập nhật danh sách điểm danh thời gian thực giữa Windows Forms và Android mà không cần người dùng bấm làm mới.
2. Triển khai Room Database trên Android:
   Lưu trữ ngoại tuyến danh sách tài liệu English for IT và sự kiện đã tải để sinh viên xem lại khi không có mạng.
3. Bổ sung bộ kiểm thử E2E tự động:
   Viết Playwright kiểm thử Web và Espresso kiểm thử Android để tích hợp vào quy trình CI/CD hoàn chỉnh.

CHÂN THÀNH CẢM ƠN THẦY VÀ CÁC BẠN ĐÃ LẮNG NGHE BÀI BÁO CÁO CỦA NHÓM VISIONARY API!

Nhóm chúng em rất mong nhận được những câu hỏi phản biện và đóng góp quý báu từ Thầy để hoàn thiện sản phẩm.